

Internal Sampled-Data Quadrature Specification

N2.1: Validated Length-Preserving Cumulative Trapezoidal Integration

mdstats

2026-07-14

Contents

Purpose and implementation stage	1
Borrowed method and mdstats contribution	1
Borrowed numerical method	1
mdstats-specific contract	1
Public status and dependency boundary	2
Function specification	2
Inputs	2
Output	2
Algorithm	3
Numerical properties	3
Accuracy	3
Length preservation	3
One-sample input	3
Nonuniform coordinates	3
Complexity	3
Invalid and edge cases	3
Test requirements	4
Reuse plan	4
References	4

Purpose and implementation stage

This document specifies the private module

`mdstats/analysis/_quadrature.py`

for roadmap stage N2.1. The implemented function is

`cumulative_trapezoid_zero(values, coordinates, *, axis=0)`

It provides one validated sampled-data integration primitive for later Green-Kubo transport functions. It does not interpret the physical meaning of its input.

Borrowed method and mdstats contribution

Borrowed numerical method

The arithmetic is the composite trapezoidal rule. For samples (x_j, y_j) with strictly increasing coordinates,

$$I_k = \sum_{j=0}^{k-1} \frac{y_j + y_{j+1}}{2} (x_{j+1} - x_j).$$

The implementation delegates this arithmetic to `scipy.integrate.cumulative_trapezoid` [1]. The trapezoidal rule and SciPy routine are borrowed numerical machinery; they are not derived by mdstats.

mdstats-specific contract

The following behavior is an mdstats design:

- coordinates must be finite, one-dimensional, and strictly increasing;
- values must be finite;
- the coordinate count must equal the selected integration-axis length;
- computation is performed in `float64`;
- the returned array has exactly the same shape as the input values;
- the first sample on the integration axis is set exactly to zero;
- the function never sorts, deduplicates, smooths, interpolates, or extrapolates input data;
- validation errors are raised before SciPy is called.

Public status and dependency boundary

The module is private. Public analysis functions consume it internally.

It depends on:

NumPy
SciPy `integrate`

It does not depend on `VACFResult`, `MSDResult`, trajectory collections, selection machinery, or spectral modules. This makes it reusable for later current-correlation and stress-correlation integrals.

Function specification

```
def cumulative_trapezoid_zero(
    values: ArrayLike,
    coordinates: ArrayLike,
    *,
    axis: int = 0,
) -> NDArray[np.float64]:
    ...
```

Inputs

values

A real sampled array with at least one dimension. Any numeric input accepted by `numpy.asarray` is converted to `float64`.

The selected axis may have one or more samples. Other axes may represent components, species, atoms, tensor entries, or independent correlation functions.

coordinates

A one-dimensional coordinate array satisfying:

$$N_x = \text{shape}(\text{values})[\text{axis}],$$

and, for every adjacent pair,

$$x_{j+1} > x_j.$$

Uniform spacing is not required.

axis

An integer axis index. Negative indices are accepted using normal Python/NumPy axis semantics. Boolean values are rejected even though `bool` is an integer subclass in Python.

Output

The output is a `float64` array with the same shape as `values`.

For every fixed index tuple on the nonintegration axes,

$$\text{result}[0] = 0,$$

and the remaining samples are cumulative trapezoidal integrals on the provided coordinate grid.

Algorithm

1. Convert values to float64.
2. Require values.ndim >= 1.
3. Normalize and validate axis.
4. Convert coordinates to one-dimensional float64.
5. Require at least one coordinate.
6. Require coordinate count == values.shape[axis].
7. Reject nonfinite values or coordinates.
8. Reject repeated or decreasing coordinates.
9. Call `scipy.integrate.cumulative_trapezoid` with `initial=0.0`.
10. Force the first output slice to exact floating-point zero.
11. Return the float64 array.

Numerical properties

Accuracy

For smooth data on a uniform grid with step h , the composite trapezoidal rule has global deterministic quadrature error of order

$$O(h^2).$$

For correlation functions estimated from finite molecular-dynamics trajectories, sampling noise and tail uncertainty usually dominate this local quadrature error. Higher-order quadrature is therefore not automatically more physically accurate.

Length preservation

SciPy normally returns one fewer sample unless `initial` is supplied. `mdstats` always uses

`initial=0.0`

so that transport curves align exactly with the original lag coordinates.

One-sample input

For one coordinate and one sample, there is no interval to integrate. The result is a same-shaped zero array.

Nonuniform coordinates

The method uses each exact interval width

$$\Delta x_j = x_{j+1} - x_j.$$

No uniform-grid approximation is introduced.

Complexity

Let M be the total number of scalar values. Time complexity is

$$O(M),$$

and the output requires $O(M)$ memory. The implementation does not allocate an interpolated or refined coordinate grid.

Invalid and edge cases

The function rejects:

- scalar values;
- multidimensional coordinates;
- empty coordinates;
- shape mismatch;
- NaN or infinite values;
- NaN or infinite coordinates;
- repeated coordinates;

- decreasing coordinates;
- noninteger axes;
- out-of-range axes.

It does not reject negative coordinates because many mathematical integrals can legitimately use them. Physical callers impose their own coordinate semantics.

Test requirements

The implementation is accepted only if tests cover:

1. constant data on a uniform grid;
2. a nonuniform grid against SciPy directly;
3. multidimensional arrays and negative axes;
4. exact first-sample zero;
5. one-sample input;
6. `float32` input converted to `float64`;
7. no mutation of inputs;
8. every invalid case listed above.

Reuse plan

N2.1 is reused by:

- GK1 running self diffusion from VACF;
- GK4 optional VACF-to-MSD reconstruction;
- CC3 ionic-conductivity integration;
- later stress-correlation transport functions.

It is intentionally separate from `_spectral.py`. Spectral bin summation and sampled time integration have different measures and normalization rules.

References

[1] P. Virtanen, R. Gommers, T. E. Oliphant, et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods* **17**, 261-272 (2020). DOI: 10.1038/s41592-019-0686-2.